OPERATING SYSTEMS LAB Practice Questions

1. Create the directory structure os_practical/{input,output,scripts,archive} using a single command. (2 Marks)

2. Inside scripts, create run.sh. Set permissions so that the owner can read, write, and execute the file, while the group and others can only read it. Verify the permissions using ls -l. (3 Marks)

3. Create and implement the following Python program:

- Create scripts/process_demo.py using os.fork(), and handle fork failure.
- The child must print its PID and parent PID, then terminate with exit status 5.
- The parent must print its PID and child PID, wait for the child, and display the child's exit status. (6 Marks)

4. Write a Python program using os.fork(), os.exec*(), and os.waitpid() to:

- i. Display the current directory and the contents of the archive directory in long format.
- ii. Display disk-space usage in human-readable format and system uptime.
- iii. Display the current date and time.
- Use one child per command, wait for completion before continuing, and handle fork/exec errors. (9 Marks)

Conditions:

- Use fork() to create a child process for each command.
- Execute Linux commands using an appropriate exec() family function (in Python, use the os.exec*() family).
- Use wait() or waitpid() to ensure each command finishes before the next one starts.
- Display a suitable error message if process creation or command execution fails.

Note: os.fork() and os.exec() are intended for Unix/Linux environments such as Ubuntu.*

OPERATING SYSTEMS LAB Practice Questions

1. Create the directory structure systemlab/{data,logs,results,backup} using a single command. (2 Marks)

2. Create logs/system.log. Set its permissions to read/write for the owner, read-only for the group, and no permissions for others. Display the final permissions using ls -l. (3 Marks)

3. Create and implement the following Python program:

- Create data/parent_child.py using os.fork(), including error handling if fork fails.
- The child prints "Child running" with its PID and parent PID, then exits with status 3.
- The parent prints its PID and the child PID, waits for the child, and reports the exit status. (6 Marks)

4. Write a Python program using os.fork(), an os.exec*() function, and os.wait() or os.waitpid() to:

- i. Show the current working directory and list files, including hidden files, in long format.
- ii. Display memory usage and the current user's running processes.
- iii. Display the number of seconds the system has been running (uptime).
- Create a child for each command, wait for each child sequentially, and show an error message if a command cannot be executed. (9 Marks)

Conditions:

- Use fork() to create a child process for each command.
- Execute Linux commands using an appropriate exec() family function (in Python, use the os.exec*() family).
- Use wait() or waitpid() to ensure each command finishes before the next one starts.
- Display a suitable error message if process creation or command execution fails.

Note: os.fork() and os.exec() are intended for Unix/Linux environments such as Ubuntu.*

OPERATING SYSTEMS LAB Practice Questions

1. Create the directory structure linux_exam/{documents,temporary,programs,logs} using a single command. (2 Marks)

2. Create documents/notes.txt. Set permissions so the owner has read/write access, the group has read-only access, and others have read-only access. Verify using ls -l. (3 Marks)

3. Create and implement the following Python program:

- Create programs/fork_wait.py. Use os.fork() and handle a possible fork error.
- The child prints its PID and parent PID, then exits with status 7.
- The parent prints its PID and child PID, waits for the child, and prints the decoded exit status. (6 Marks)

4. Write a Python program using os.fork(), os.exec*(), and os.waitpid() to:

- i. Print the current working directory and list the contents of documents in detailed format.
- ii. Display disk usage for mounted file systems and memory usage in human-readable format.
- iii. Display the current date and time.
- Run each command in a separate child process, wait for it to finish before continuing, and handle command-execution failures. (9 Marks)

Conditions:

- Use fork() to create a child process for each command.
- Execute Linux commands using an appropriate exec() family function (in Python, use the os.exec*() family).
- Use wait() or waitpid() to ensure each command finishes before the next one starts.
- Display a suitable error message if process creation or command execution fails.

Note: os.fork() and os.exec() are intended for Unix/Linux environments such as Ubuntu.*

OPERATING SYSTEMS LAB Practice Questions

1. Create the directory structure labwork/{source,backup,output,monitoring} using a single command. (2 Marks)

2. Inside monitoring, create monitor.log. Set permissions to rw-r----- (owner read/write, group read-only, others no access) and show the result using ls -l. (3 Marks)

3. Create and implement the following Python program:

- Create source/fork_status.py using os.fork(), and handle the case where fork fails.
- The child prints "Child process started" with its PID and parent PID, then exits with status 2.
- The parent prints its PID and the child PID, waits for the child, and displays the exit status. (6 Marks)

4. Write a Python program that uses os.fork(), an os.exec*() function, and os.wait() or os.waitpid() to:

- i. Display the present working directory and list all entries, including hidden files, in long format.
- ii. Display system uptime and disk-space usage in human-readable format.
- iii. Display processes belonging to the current user and the current date/time.
- Use a separate child process for each command, wait for each child sequentially, and handle fork/exec errors. (9 Marks)

Conditions:

- Use fork() to create a child process for each command.
- Execute Linux commands using an appropriate exec() family function (in Python, use the os.exec*() family).
- Use wait() or waitpid() to ensure each command finishes before the next one starts.
- Display a suitable error message if process creation or command execution fails.

Note: os.fork() and os.exec() are intended for Unix/Linux environments such as Ubuntu.*

OPERATING SYSTEMS LAB Practice Questions

1. Create the directory structure final_lab/{reports,logs,programs,backup} using a single command. (2 Marks)

2. Create reports/final.txt. Change its permissions so that the owner has read/write/execute access, the group has read access, and others have no access. Verify using ls -l. (3 Marks)

3. Create and implement the following Python program:

- Create programs/child_info.py using os.fork(), including handling for fork failure.
- The child prints its PID and parent PID and exits with status 4.
- The parent prints its PID and child PID, waits for the child, and displays the child's exit status. (6 Marks)

4. Write a Python program using os.fork(), an appropriate os.exec*() function, and os.wait() or os.waitpid() to:

- i. Display the current working directory and list all files, including hidden files, with detailed information.
- ii. Display memory and disk-space usage in human-readable format.
- iii. Display current-user processes and the current date/time.
- Create a separate child process for each command, wait for completion before proceeding, and display suitable error messages for failed command execution. (9 Marks)

Conditions:

- Use fork() to create a child process for each command.
- Execute Linux commands using an appropriate exec() family function (in Python, use the os.exec*() family).
- Use wait() or waitpid() to ensure each command finishes before the next one starts.
- Display a suitable error message if process creation or command execution fails.

Note: os.fork() and os.exec() are intended for Unix/Linux environments such as Ubuntu.*